



ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

FILAMENT CONTROLLER

Albian Salihu

5th January 2024

ACKNOWLEDGEMENTS

My heartfelt gratitude goes to the EPFL spacecraft team and UniBe for their crucial role in bringing this exciting project to fruition. Special thanks to Dr. Fausch Rico for his warm welcome and invaluable assistance throughout the project. I'm deeply appreciative of Trimble Scott's patience and detailed explanations. David Reis deserves acknowledgment for his explanation of various project aspects and efforts in repairing the testing board. Lastly, I extend my thanks to Alex Iseli for his meticulous review of the report and for sharing his knowledge about the filament and the mass spectrometer.

Contents

1	Introduction	3
1.1	Context	3
1.2	Goals	4
1.3	Project specifications	4
1.3.1	Participants	4
1.3.2	Project description	4
1.3.3	Tasks and milestones	5
2	Design Concept	6
2.1	Description of the schematic	6
2.2	Open Loop Controller	8
2.3	Implementation of the open loop controller in VHDL	9
2.4	Implementation of the filament controller in an FPGA system	10
2.5	Testbench of Open Loop controller	11
2.6	Closed Loop Controller	11
2.6.1	Choice of controller	11
2.6.2	PID Calculations	12
2.6.3	PID implementation in VHDL	13
2.6.4	Two PID controllers to account for fast and slow variations	13
2.6.5	Final Filament controller	14
2.7	Simulation setup	15
3	Results	18
3.1	Testbench result of the open Loop	18
3.2	Hardware result of open loop	18
3.3	Results of closed loop	19
4	Discussion	20
4.1	Disscussion	20
4.2	Improvements	20
5	Conclusion	21
6	Appendix	22
6.1	User guide to test the filament controller	22
6.1.1	Opening the project on vivado	22
6.1.2	Changing the value of the phase shift for testing	23
6.1.3	Programming the FPGA	27

CHAPTER 1

INTRODUCTION

1.1 CONTEXT

The CHESS mission, standing for "Constellation of High-performance Exospheric Science Satellites," is designed to analyze the exosphere's chemical makeup. Its equipment includes a Time of flight mass spectrometer designed to fit inside a cubesat (CubesatTOF, which is the same as Föhn et al. 2021 and Fausch, Zimmermann et al. 2023) and a global navigation satellite system (GNSS). The mission's goals are to catalog chemical species in the exosphere and upper ionosphere, study the dynamics of these species and total electron content, integrate this data with other Earth observation resources for real-time upper atmosphere monitoring, and develop the necessary space and ground segments for these measurements.

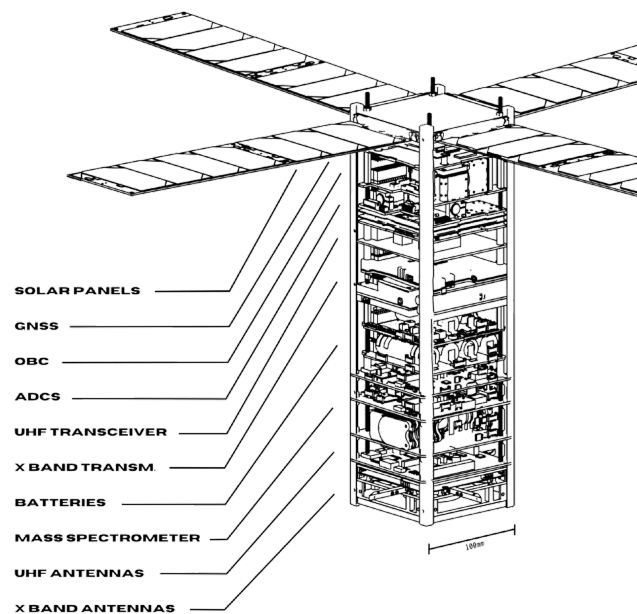


FIGURE 1.1
Satellite layout

The mass spectrometer is positioned at the bottom of the satellite (Figure 1.1) just above the UHF and x-band antennas. The cubesatTOF (Figure 1.2) is composed of 3 boards¹. Starting from the top there is the power board, the sensor board and the digital board. The main focus of this project is on the power board where the filament is controlled from. The field programmable gate arrays (FPGA) which holds the filament controller logic is connected to the digital board but the outputs drive the FIL_SYNC and FIL_PHASE pins on the power board. The schematics of the power board is shown in section 2.1 where it will be further described.

1.2 GOALS

Building on the intricate design elements in the CHESS mission, the project goals are twofold. Firstly the project aim is to design a filament controller that effectively regulates the phase alignment between sync and phase signals, either in response to the emitted current (closed-loop) or based on user input (open-loop). This dual-mode operation allows for both responsive and predetermined control settings. More detail about the project in subsection 1.3.2. Secondly the goal of the project is to collaborate with a team of engineers and to improve presentation and communication skills.

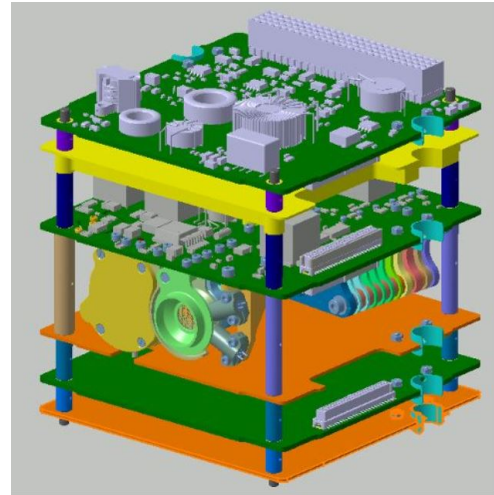


FIGURE 1.2
CubesatTOF composition

1.3 PROJECT SPECIFICATIONS

1.3.1 PARTICIPANTS

The realization of the project goals hinges on the expertise and collaboration of a diverse team including :

- Leading supervisor : Prof Dr. Schmid Alexandre
- Supervisor : Prof. Dr. Wurz Peter
- Co-supervisor : Dr Fausch Rico
- Supervising engineer : Trimble Scott
- Candidate : Albian Salihu

1.3.2 PROJECT DESCRIPTION

Delving deeper into the technical aspects, the CubeSatTOF mass spectrometer was selected to analyse the chemical composition of Earth's upper atmosphere on board the student-driven Constellation of Highperformance Exosphere Science Satellites (Fausch, Moeller et al. 2022) mission. The CHESS mission is part of EPFL's initiative to engage students in applied space projects. The project is designed to solve a 40-year-old puzzle in atmospheric science using a mass spectrometer and a GNSS instrument. The mass spectrometer requires novel concepts for the data processing unit to allow for miniaturization, enabling this CubeSat-type mission. The data processing unit of the CubeSatTOF instrument is currently in an early prototype stage (Xilinx Kria K26, System on Module). Currently, the team is aiming for

¹The green boards shown in Figure 1.2

developing a fully functional data processing unit from scratch, making use of the existing boards within this multi-organization, ESA-supported project. Electron emitters are the most delicate subsystem of a mass spectrometer. In fact, even modern mass spectrometers carry redundant cathodes, as they are used to ionise the neutral gas of interest to manipulate the trajectory of species with electrostatic lenses in the time-of-flight instruments. Despite the recent successful near-Earth commissioning of the cathodes of the NIM instrument on board ESA's large-class JUICE mission, their commissioning and operation procedure can be optimized. In this framework, the student's goal is to provide stable electron emission by developing and implementing a controller for operating thermionic emitters. The controller will operate inside the data processing unit and control the analogue hardware.

1.3.3 TASKS AND MILESTONES

To translate the project goals into tangible outcomes, a series of tasks and milestones have been established, as outlined below. These not only serve as a roadmap but also as benchmarks for the progress and success of the project.

For a systematic approach to the project, specific 'Tasks' have been delineated.

- Specify requirements to ensure a useful operation
- Establish a simulation tool chain
- Design controller in both open and closed loop
- Define testing needs and perform testing
- provide API for accessing data and controlling the controller²
- Provide documentation and reporting
- Partner with software and electronics engineers, project management, and systems engineering to ensure an operational system.

To effectively track the project's progress and ensure adherence to timelines, a detailed 'Milestones Table' (Table 1.1) outlines the key phases and deadlines

Milestone	Description	Weeks	Due(CW)
Kick-off	Kick-off	-	38
M1	Specify requirements	2	40
M2	Design open loop controller	3	43
M3	Define test plan for comissioning	1	44
M4	Verify open loop controller on hardware	1	45
M5	Establish simulation tool chain	2	47
M6	Design closed loop controller and test it on a rolling basis	2	49
M7	Test closed loop controller on hardware	2	51
M8	Implement API to modify FPGA controller	1	51
end	Project end	-	51

TABLE 1.1
Milestones table

²bonus

CHAPTER 2

DESIGN CONCEPT

2.1 DESCRIPTION OF THE SCHEMATIC

Before beginning to explain the designing of the filament controller, there is a need to describe how the filaments are controlled. The whole schematic of the power board's filament controller is shown in Figure 2.2. The FIL_PHASE and FIL_SYNC (left of Figure 2.2) are the outputs of the FPGA phase and sync signals. The square signals (that have only 2 states, 0 and +3.3V) are then converted to a tri-state format (+12, 0 and -12 like shown in Figure 2.1) which is used to power the filament (located on the right of Figure 2.2). Parallel to that there is an emission current measurement made to allow for closed loop control (top of Figure 2.2 in the orange box called "Emission current measurement").

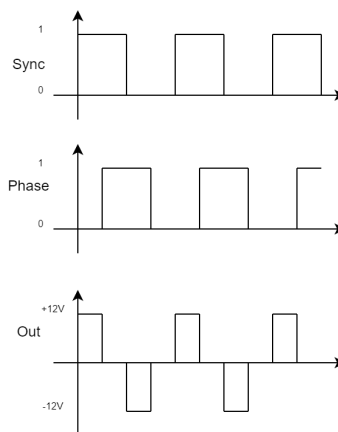


FIGURE 2.1
How the sync and phase are transformed

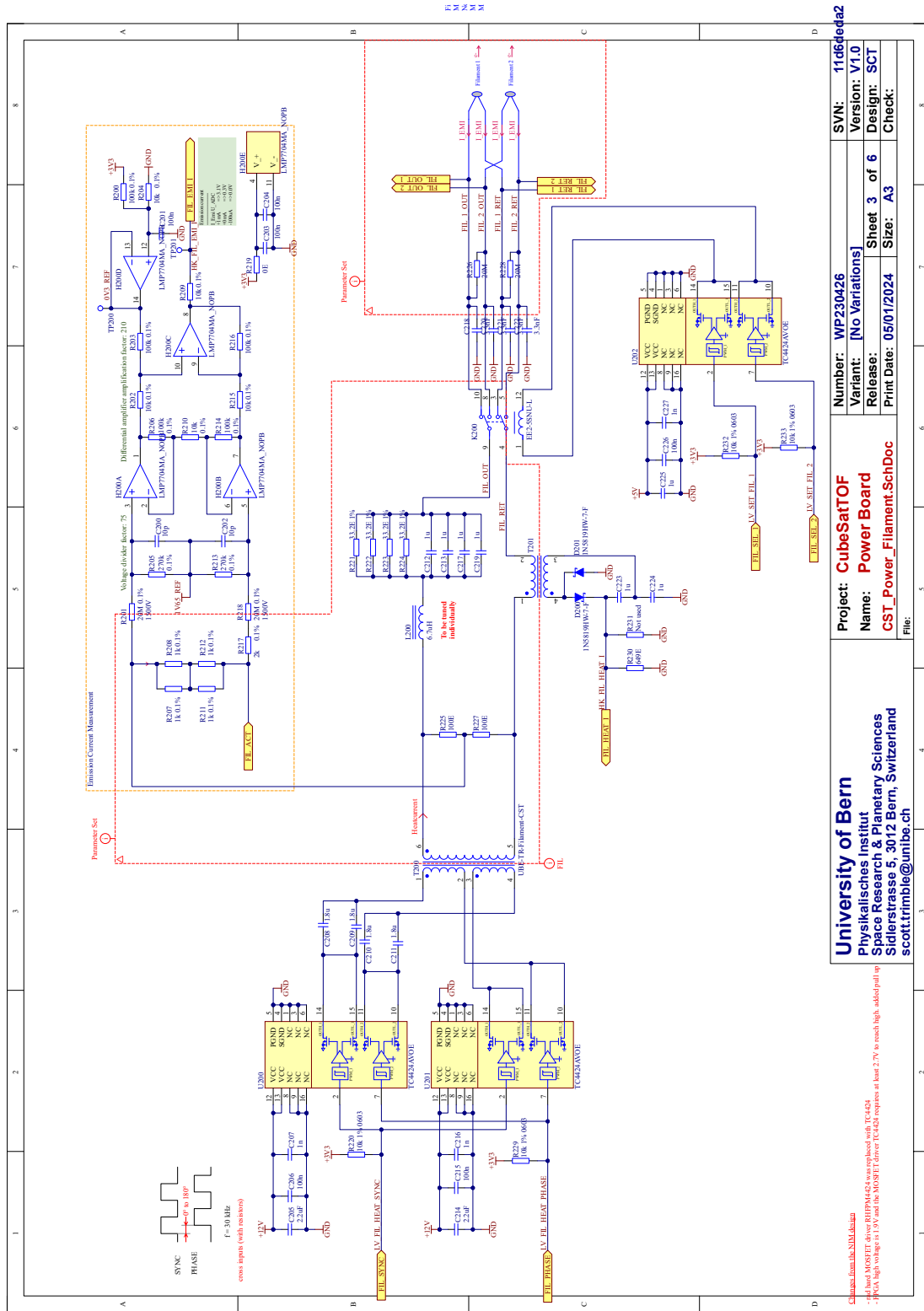


FIGURE 2.2
Schematic of the power board's filament controller

2.2 OPEN LOOP CONTROLLER

In general, in VHDL the best way to describe how an code works is through state machine diagrams. The code is more comprehensible and to code such diagrams is straight forward. That is the reason why most of the explanations in the next chapters are done using state machines.

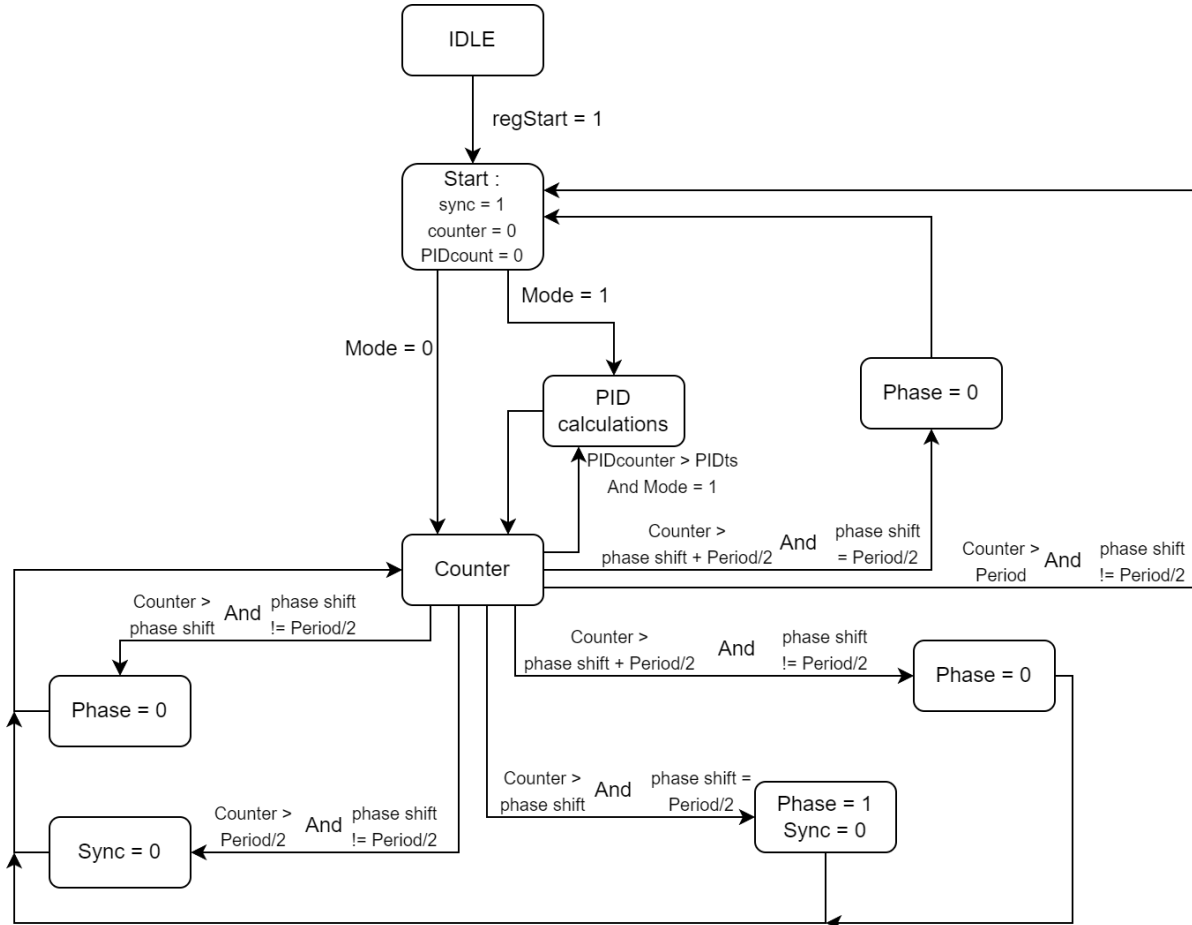


FIGURE 2.3
Final state machine of the controllers

The filament controller's state machine, as detailed in Figure 2.3, initiates in an IDLE state. This initial phase ensures all registers and any other processes are fully prepared before commencing signal generation. The active phase commences once the regStart register switches to 1, triggering signal outputs to the phase and sync pins. Details on the PID calculations are covered in section 2.6. The Counter state forms the crux of the controller, where it engages in counting, comparison of the counter's value, and executes required actions based on the counter comparisons, guided by Figure 2.4:

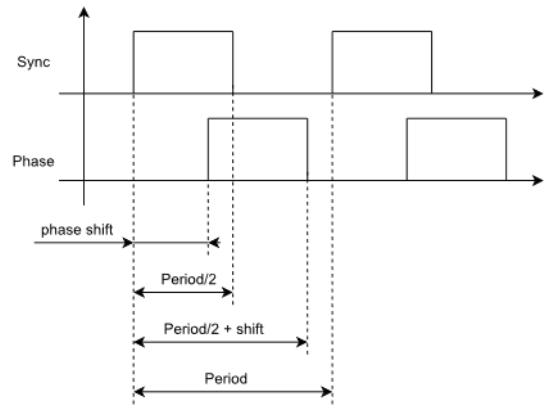


FIGURE 2.4
Description of the phase and sync signals

Within the counter state of the state machine, two primary hidden states exist. The first occurs when the phase shift is less than half of the period, where the phase signal is set to 1 as the counter hits the phase shift value and the sync signal is reset to 0 upon reaching half the period. In the second state, where the phase shift equals half the period, the phase signal is reset after the counter hits the sum of half the period and phase shift. The cycle completes with the sync signal set to 1 at the period value and the counter resetting to 0.

In the second state, when the phase shift is exactly half the period, the control system behaves distinctively. Upon the counter reaching the phase shift value, it simultaneously resets the sync signal to 0 and sets the phase signal to 1. Conversely, when the counter hits the period value, it inversely resets the phase signal and sets the sync signal to 1, also resetting the counter. This mechanism is crucial to prevent errors in cases where the phase shift is precisely half the period.

2.3 IMPLEMENTATION OF THE OPEN LOOP CONTROLLER IN VHDL

The purpose of this subsection is not to have an in depth description of the VHDL¹ code but only some parts that are worth explaining.

IMPLEMENTATION OF THE PHASE SHIFT

The filament controller is designed to be used in a C code in vitis, in a user friendly manner. To perform that, the input asked for the desired phase shift is in degrees [°]. There needs to be a conversion from degrees to number of cycles to be able to count the necessary phase shift. The calculation is described in Equation 2.1 :

$$phaseshift = \frac{desFreq \times angle}{360} \quad (2.1)$$

With the number of cycles for the desired phase shift $phaseshift$, the desired frequency of the output signals in phase and sync pin in cycles $desFreq$ and the user input phase shift angle in degrees $angle$.

In order not to have timing violations during synthetizing step the Equation 2.1 should be pipelined. Pipelining in VHDL means dividing a complex task into smaller, more manageable stages. Each stage completes a part of the task and passes its output to the next stage. The pipelining of this operation consist

¹The language used for the FPGA interfaces

of four steps. First the phase shift value is stored in a temporary variable, then the result of $\frac{desFreq}{360}$ is stored in another temporary variable which is the operation that takes the most time. Then the result of the multiplication between the two temporary variables is stored in a third temporary variable which is then converted to the desired variable type (std logic vector in that case).

IMPLEMENTATION OF THE DESIRED FREQUENCY

The user input for the desired frequency is in KHz. That value needs to be changed to number of cycles and to do that we need to have the clock frequency of the clock used in the FPGA system. After that the calculation is straight forward and shown in Equation 2.2:

$$desFreq = \frac{CPUFreq}{Freq} \quad (2.2)$$

With the desired frequency of the output sync and phase pins in cycles $desFreq$, the clock connected to the filament controller in KHz $CPUFreq$ and the desired user frequency in KHz $Freq$.

2.4 IMPLEMENTATION OF THE FILAMENT CONTROLLER IN AN FPGA SYSTEM

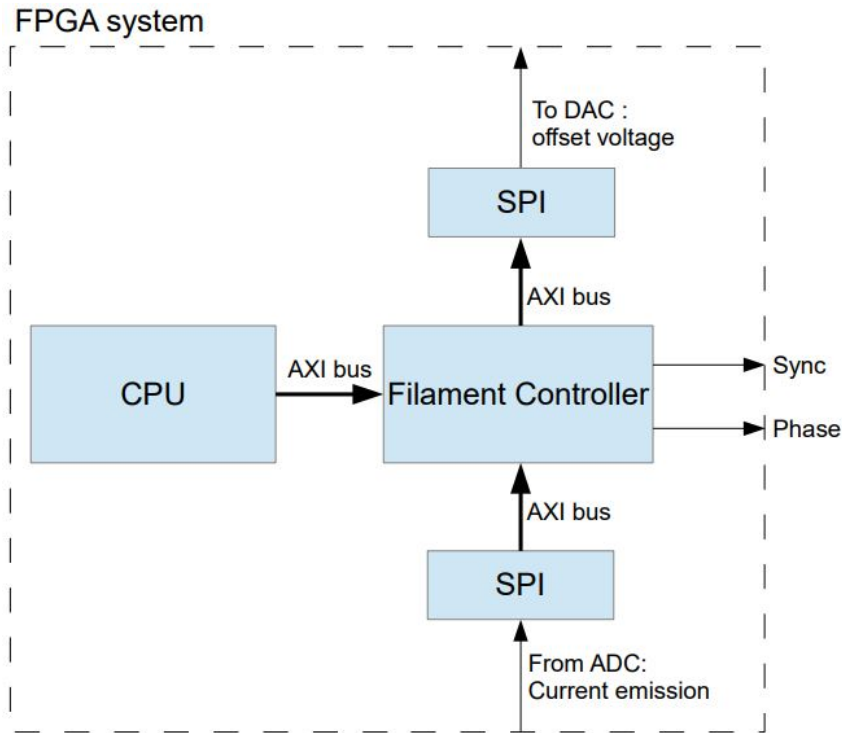


FIGURE 2.5
Fpga full system

After creating the filament controller, it needs to be included in an FPGA system in order to work with the clock of the CPU and potentially other interfaces. To be able to get the values of the emitted current and to send the offset voltage needed for the filament to work, the system needs to implement SPI protocols interfaces. The final system would have looked like Figure 2.5, unfortunately the spi connections in the

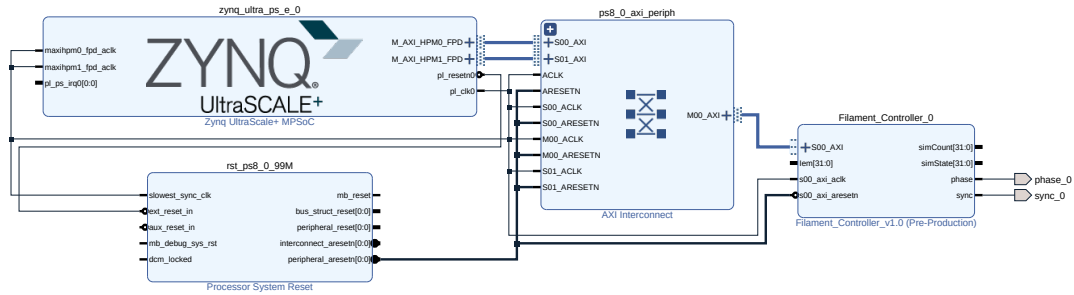


FIGURE 2.6
Vivado FPGA system

filament controller interface has not been made because of a lack of time. The final system that has been tested with the hardware is shown in Figure 2.6 (Vivado block design):

2.5 TESTBENCH OF OPEN LOOP CONTROLLER

Testing the Open Loop controller involves verifying its ability to produce the desired phase shift between the sync and phase signals, with particular attention to the special case of a 180° phase shift. The testbench setup for the open loop controller includes creating clock generation, implementing an asynchronous reset, and procedures to input signals into the filament controller. During the main simulation process, various phase shifts are outputted after invoking the asynchronous reset, ensuring the controller's functionality across different scenarios.

2.6 CLOSED LOOP CONTROLLER

2.6.1 CHOICE OF CONTROLLER

The emission current is fairly stable with time and a simple PID has been chosen since it is the easiest controller to implement. The other candidate was a two stage PID which is described in subsection 2.6.4. The two-stage PID has not been chosen because it is more complex and at that stage of the project, there is no need of a more complex controller. Further discussion on the controller's choice is made under the section 4.2.

2.6.2 PID CALCULATIONS

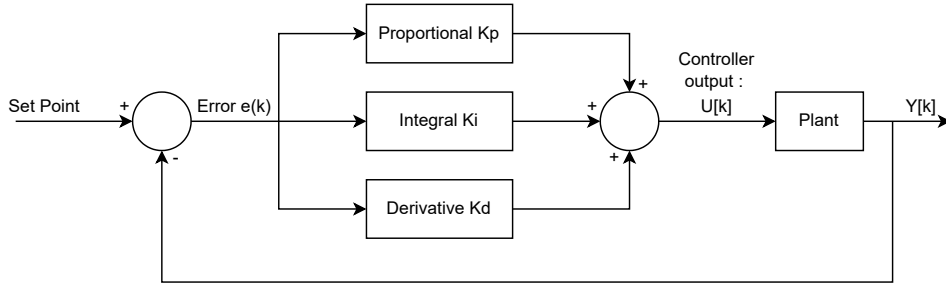


FIGURE 2.7
PID framework

For the task needed a simple PID framework has been chosen (Figure 2.7), with the proportional K_p coefficient, the integral K_i coefficient and the derivative K_d coefficient. The calculation for such controller is Equation 2.3:

$$U[k] = K_p e[k] + K_i T_s \sum_{i=0}^k e[i] + K_d \frac{e[k] - e[k-1]}{T_s} \quad (2.3)$$

With the proportional K_p , Integral K_i , derivative coefficient K_d , the error between the desired emission current and the actual emission current e , the sampling period T_s and the time step k . As explained in subsection 2.6.1, the filament does not vary much, thus, a derivative term, an integral term or both might be useless. In such a case, and for tuning purposes, the filament controller has registers that holds the value of K_p , K_i and k_d , and to cancel a coefficient a simple 0 has to be written in the needed register.

2.6.3 PID IMPLEMENTATION IN VHDL

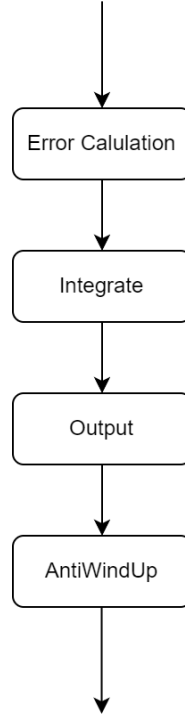


FIGURE 2.8
The state machine of the PID calculations

The state machine (Figure 2.8) starts by simply calculating the error between the desired emission current and the actual emission current. The through each iteration there is an internal signal that stores the value of the error and adds the current error to the stored one which represents the integral part of the equation. Then, the state machine goes into the output state which does the following calculation :

$$phaseshift = regKp \times Error + regKi \times PIDTs \times sumErr + regKd \times \frac{error - lasterr}{PIDTs} \quad (2.4)$$

The *lasterr* variable in Equation 2.4 is simply the stored error of the last iteration of the state machine. After the calculation is done the current *error* variable becomes the *lasterr*. Finally to protect the system and not to output a phase shift that is out of bound there is an antiwindup stage that simply crops the desired values of phase shift to stay between 0° and 180° .

2.6.4 TWO PID CONTROLLERS TO ACCOUNT FOR FAST AND SLOW VARIATIONS

Another candidate for the controller could be a two-stage PID controller which holds twice as many parameters as the simple PID but which works basically in the same way. There is a controller for the fast dynamics and on for the slow dynamics. Both are then added together to give the final controller output. This controller has not been chosen, however researches on this topic has been made during the project and the following equations are proposed if the simple PID is not sufficient :

Fast dynamics of PID controller :

$$U_{fast}[k] = K_{p,fast}e[k] + K_{i,fast}T_{s,fast} \sum_{i=0}^k e[i] + K_{d,fast} \frac{e[k] - e[k-1]}{T_{s,fast}} \quad (2.5)$$

Slow dynamics of PID controller :

$$U_{slow}[k] = K_{p,slow}e[k] + K_{i,slow}T_{s,slow} \sum_{i=0}^k e[i] + K_{d,slow} \frac{e[k] - e[k-1]}{T_{s,slow}} \quad (2.6)$$

Combination of both PID's using weighted sum :

$$U[k] = \alpha U_{fast}[k] + (1 - \alpha)U_{slow}[k] \quad (2.7)$$

2.6.5 FINAL FILAMENT CONTROLLER

The final filament controller registers are described in Table 2.1 and the interconnections of the interface is shown in Figure 2.9. The interface takes a clock and two other advanced extensible interface (AXI) input, one from the CPU and one from the analog to digital converter (ADC). It outputs two pins, phase and sync, as well as an AXI output.

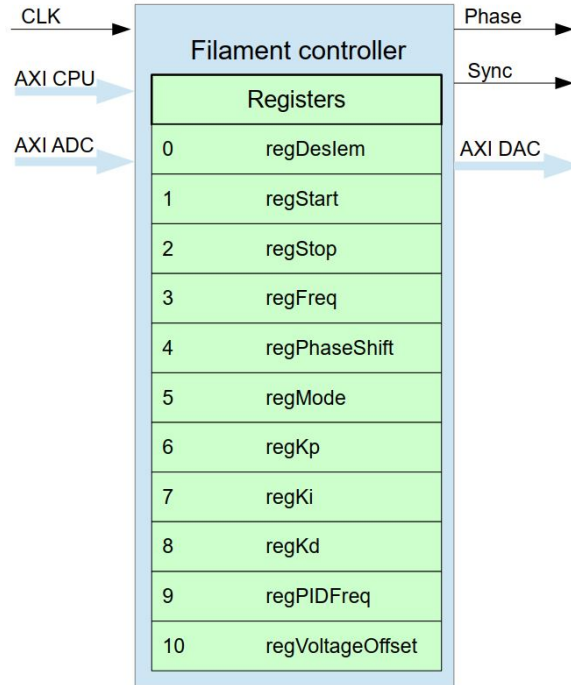


FIGURE 2.9
Register map of the filament controller

Address (hexa)	Register Name	Size (bit)	description
0x00	regDesIem	32	Holds the desired electron emission current for the closed loop
0x04	regStart	1	When set to 1, the phase and sync starts outputting square signals
0x08	regStop	1	When set to 1, pauses the system, when reset to 0 system continues where it left
0x0C	regFreq	32	Holds the desired frequency of the square signals on Phase and Sync
0x10	regPhaseShift	32	In open-loop, holds the desired value of the phase shift, in closed loops holds the actual value
0x14	regMode	1	When set to 1 mode is closed-loop, when reset to 0 mode is open-loop
0x18	regKp	32	Holds the value of the proportional controller
0x1C	regKi	32	Holds the value of the integral controller
0x20	regKd	32	Holds the value of the derivative controller
0x24	regPIDFreq	32	Holds PID's update frequency
0x28	regVoltageOffset	32	Holds the value of the needed offset voltage

TABLE 2.1
Register map of the filament controller

2.7 SIMULATION SETUP

In order to test the filament in a vacuum chamber a simulation setup has been developed by the engineers of UniBe. Figure 2.10 shows a simple diagram of the interconnection of the block on the test board. Unfortunately, this board did not work for testing and thus no tests were made in the vacuum chamber.

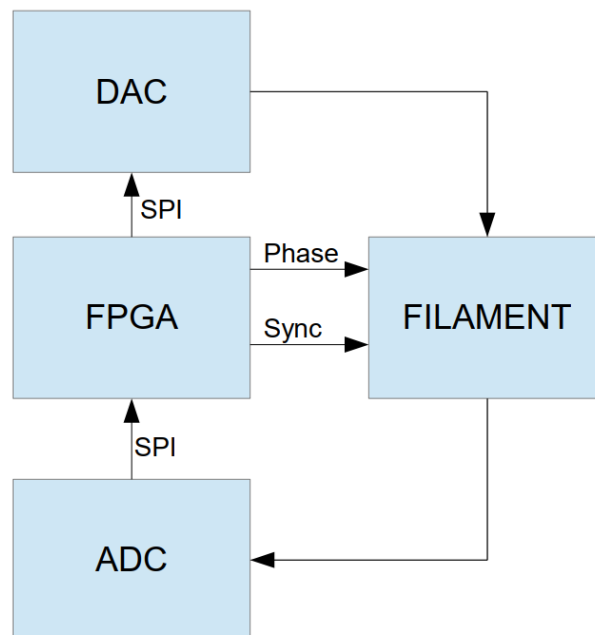


FIGURE 2.10
The simulation layout

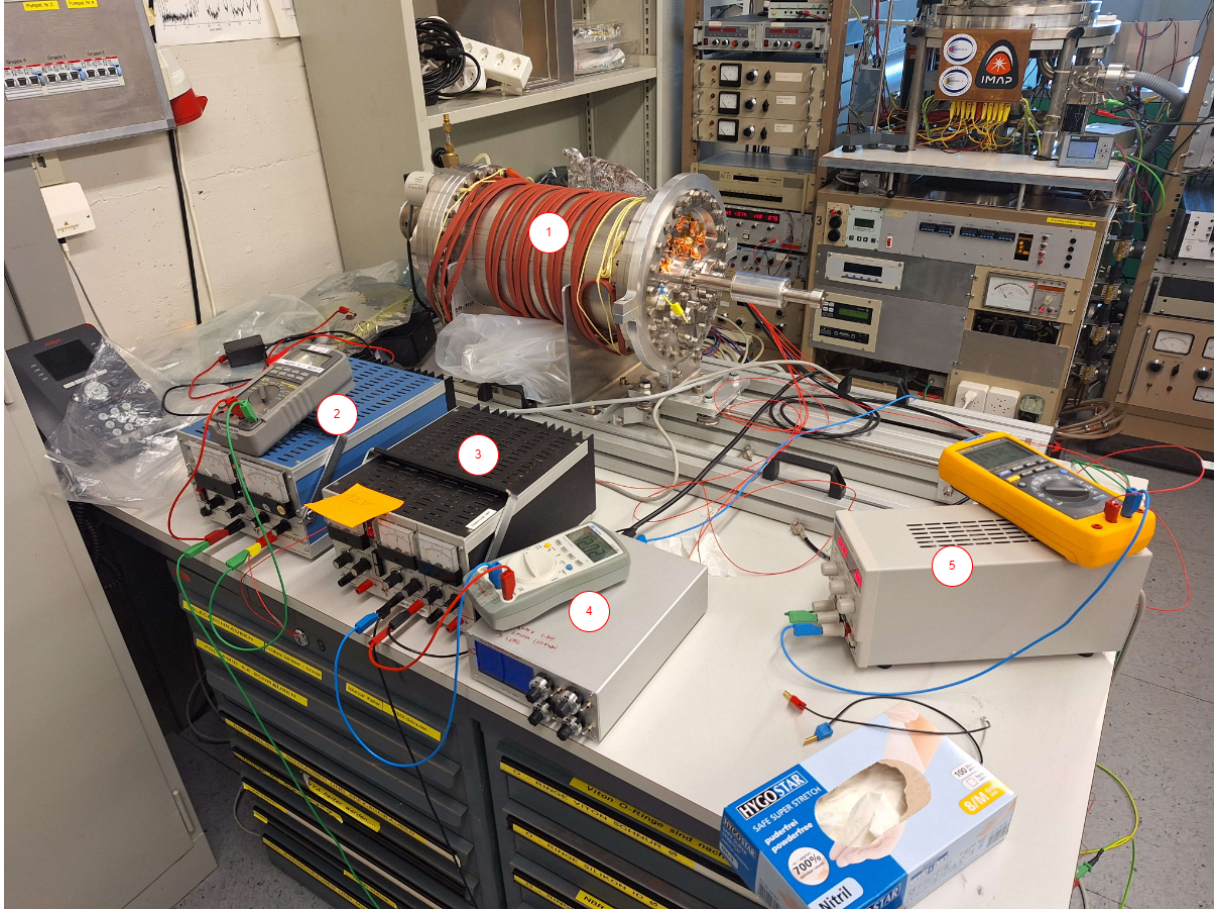


FIGURE 2.11
Lab simulation setup

Even if the simulation test could not be made due to the testing board not working, it is interesting to describe the simulation setup to understand for which setup the filament controller has been made. The simulation setup (Figure 2.11) comprises a vacuum chamber (1 in Figure 2.11) where the filament lays and four different generators. The first, blue one, "2" in Figure 2.11 is used to output a voltage to an electron repeller so that the beam of electrons is more parallel yielding better measurement of the emitted current. The black one "3" in Figure 2.11 is used to output an offset voltage to the filament for it to work properly. The one in the middle with the blue screen "4" in Figure 2.11 is what gives the voltage to the filament in place of the filament controller. It would have been replaced by the simulation board. When the electrons are emitted they are accelerated towards a lens. Once they hit the lens they produce a current that can be measured. This lens needs a certain voltage to attract the electron and yield better current measurements and that is what the last generator "5" in Figure 2.11 is used for. A quick description of the principles is described in Figure 2.12 along with a picture of the filament mounted to the vacuum chamber Figure 2.13.

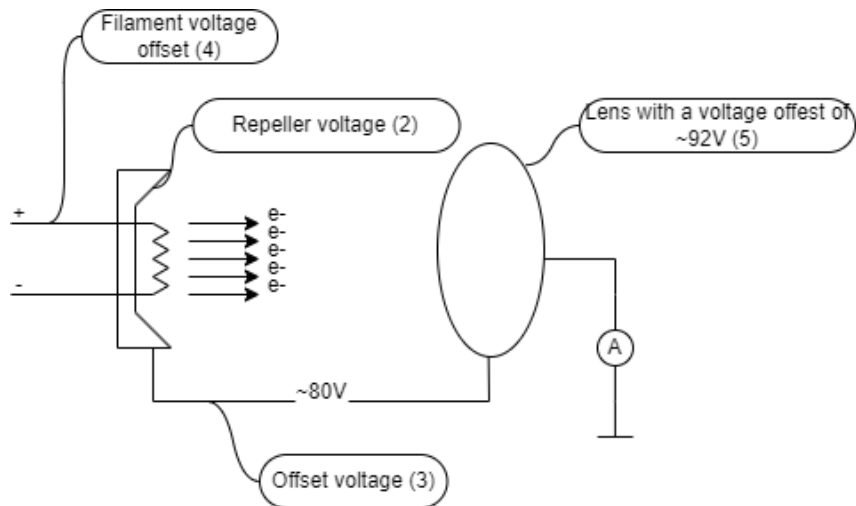


FIGURE 2.12
Description of the simulation setup

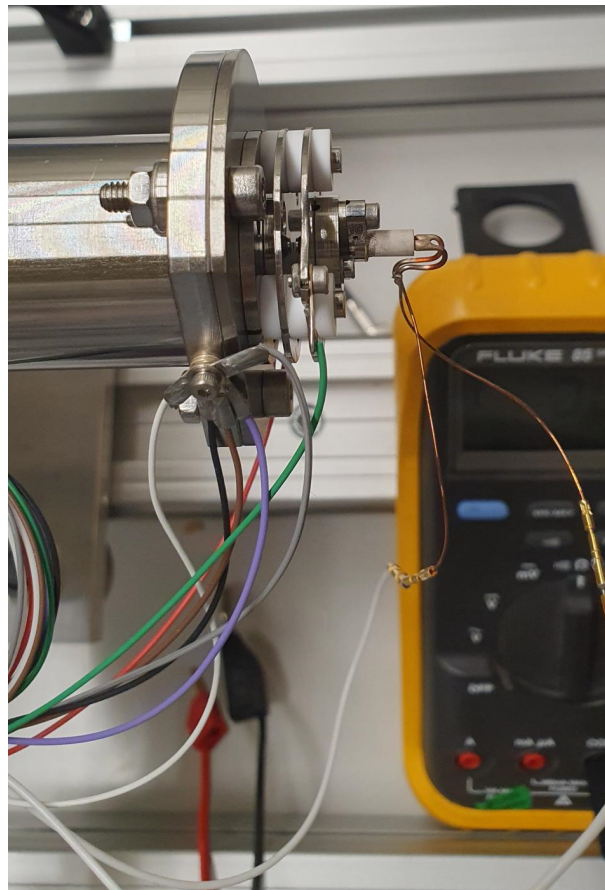


FIGURE 2.13
Filament mounted to the vacuum chamber, taken by Alex Marc Iseli

CHAPTER 3

RESULTS

3.1 TESTBENCH RESULT OF THE OPEN LOOP



FIGURE 3.1
Testbench result for a phase shift of 90°



FIGURE 3.2
Testbench result for a phase shift of 45°



FIGURE 3.3
Testbench result for a phase shift of 0°



FIGURE 3.4
Testbench result for a phase shift of 180°

3.2 HARDWARE RESULT OF OPEN LOOP

After the simulations made in the Vivado environment, the Kria k26i was programmed with the filament controller code. The results (Figure 3.5) were measured with an oscilloscope. At that stage of the project, the filament testing in the vacuum was still a goal to achieve and thus, only one phase shift configuration was tested. After that the kria k26i was unmounted from its initial board (where the hardware tests were performed) and no more results could be recorded.

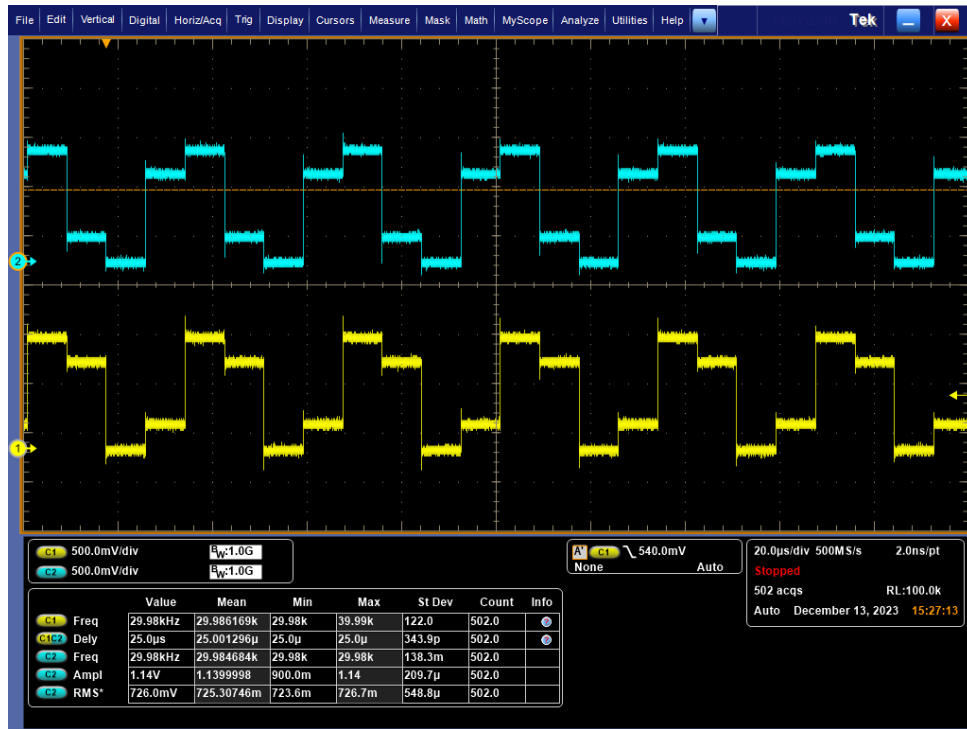


FIGURE 3.5
Result of open loop on hardware with a phase shift of 90°

The result (Figure 3.5) when configuring the output with 90° phase shift showed a coupling between the two pins. This is probably due to the way the measurements were made. The coupling should not be taken into account to interpret the results.

3.3 RESULTS OF CLOSED LOOP

It is very complicated to test the closed loop controller in simulation since a simulation of the reaction of the filament to a certain voltage input has to be made. Due to this complication, the testing of the closed loop controller was planned to be performed with the simulation setup described in section 2.7. Unfortunately even the open-loop controller could not be tested in this setup due to the testing board not working. Simulating the filaments behaviour is not recommended since it implies complicated setups.

CHAPTER 4

DISCUSSION

4.1 DISSCUSSION

Beginning with an analysis of the testbench results (Figure 3.1, Figure 3.2, Figure 3.3, and Figure 3.4), it is evident that the filament controller performs flawlessly in simulation. This is demonstrated by the accurate execution of all the phase shifts. This suggests that the filament controller is operational under all tested conditions. Furthermore, a detailed inspection of the output in Figure 3.5 reveals that, in actual hardware implementation, the filament is functioning as expected in a 90° phase shift configuration. There is a slight difference in the frequency of the phase and sync signals. This is due to integer calculation errors. To eliminate that error another clock frequency of the CPU should be chosen such that the calculation yields an integer result. An example of clock could be to chose 300MHz or a multiple of 30000.

4.2 IMPROVEMENTS

As explained during this report, the final filament controller shown in subsection 2.6.5 has not been tested. Only a simpler version, using only the open loop was tested. The first improvement would be to test the open-loop controller into the simulation test described in section 2.7. Then next step should be to implement the SPI protocols in the the filament controller interface as well as in the FPGA system as well as to complete the closed loop controller which mainly lacks a counter to for the pid calculations. A testbench of the AXI Quad SPI v3.2 has been made to try and test the interface without results.

Another area of investigation to improve the filament controller could be to implement the two stage PID described in subsection 2.6.4. Other control system types such as MPC or LQR are way too complex for the task needed but could be interesting to test because they offer more control over the system. These advanced control strategies, while more intricate, may provide enhanced precision and responsiveness, potentially leading to better performance of the filament controller under varying conditions. Continuing from this point, the integration and testing of the complete filament controller, encompassing both open and closed-loop functionalities, is a crucial next step. This comprehensive testing should include scenarios that simulate real-world operating conditions to ensure the controller's reliability and effectiveness in practical applications.

CHAPTER 5

CONCLUSION

This report has systematically explored the development and testing of a filament controller, highlighting its performance in open-loop. The successful simulation results and the preliminary hardware testing outcomes demonstrate the controller's efficacy in achieving desired phase shifts, affirming its potential for practical application. While the full capabilities of the controller, particularly in the closed-loop configuration, remain to be tested, the groundwork laid in this project provides a solid foundation for future developments. Further enhancements, such as the implementation of SPI protocols and the exploration of advanced control strategies, are anticipated to optimize its functionality.

This project has been a remarkable journey, offering a profound understanding of the dynamics involved in space project execution. The collaboration with other engineers has not only enriched the experience but also highlighted the significance of teamwork in complex projects. It has been enlightening to witness the intricate balance between theoretical design and practical application, particularly in a field as challenging as space technology. The hope is that similar exciting projects will continue to be available for other students, providing them with equally enriching opportunities to learn and grow in the field.

CHAPTER 6

APPENDIX

6.1 USER GUIDE TO TEST THE FILAMENT CONTROLLER

This user guide is made for people with little to no experience in fpga's to still be able to test the filament controller designed in the project.

6.1.1 OPENING THE PROJECT ON VIVADO

First open vivado, and click on "open project" in the Quick Start box Figure 6.1. Alternatively, click on File → Project → open.

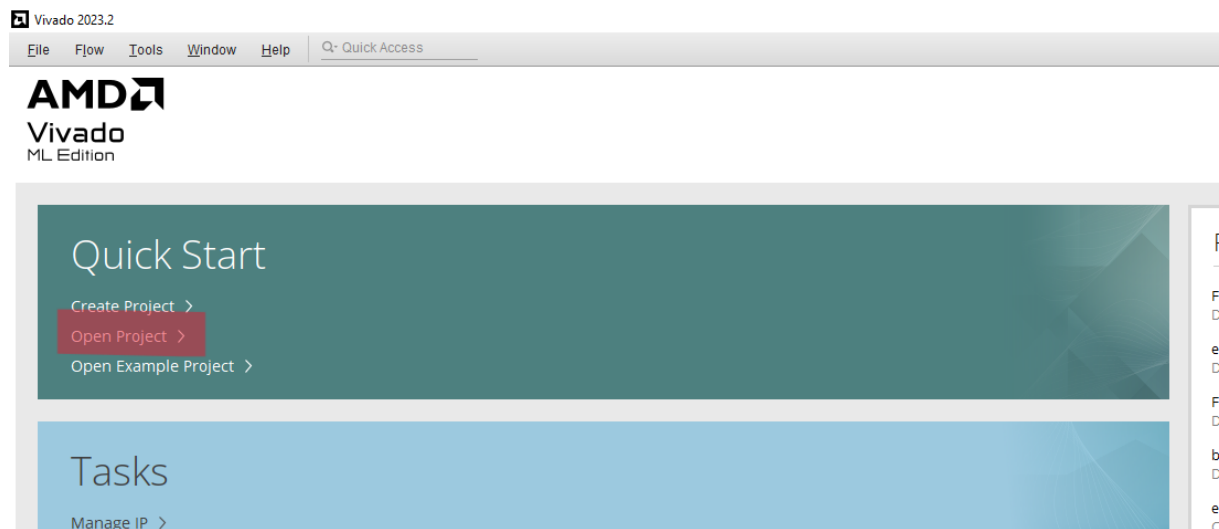


FIGURE 6.1
Image after opening vivado 2023.2

Then inside the TESTING folder, open Filament_controller_pid → Filament_controller_pid.xpr Figure 6.2.

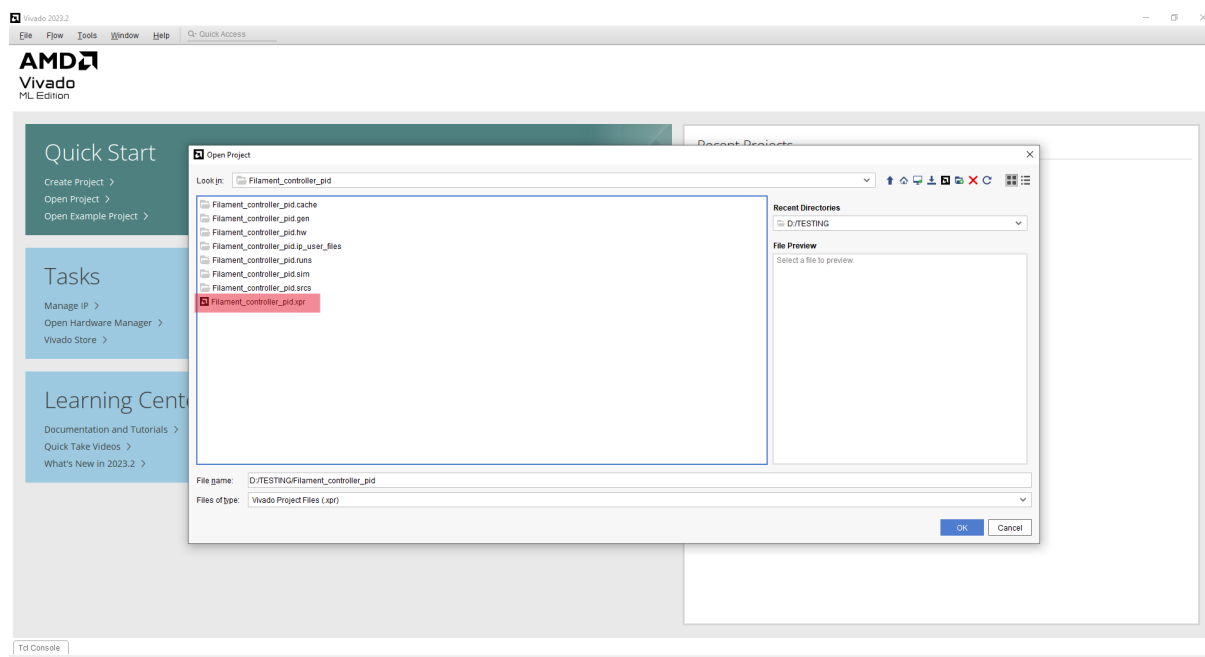


FIGURE 6.2
Opening the main project

This opens the main FPGA system from which everything will be done.

6.1.2 CHANGING THE VALUE OF THE PHASE SHIFT FOR TESTING

After opening the project, the phase shift needs to be changed. Since it is hard coded into the registers, the user has to uncomment some code and comment some other code. First go to File → Project → Open. Then inside the TESTING folder click on ip_repo → edit_Filament_Controller_v1_0.xpr Figure 6.3. And click on "No" when asked if you would like to close the project.

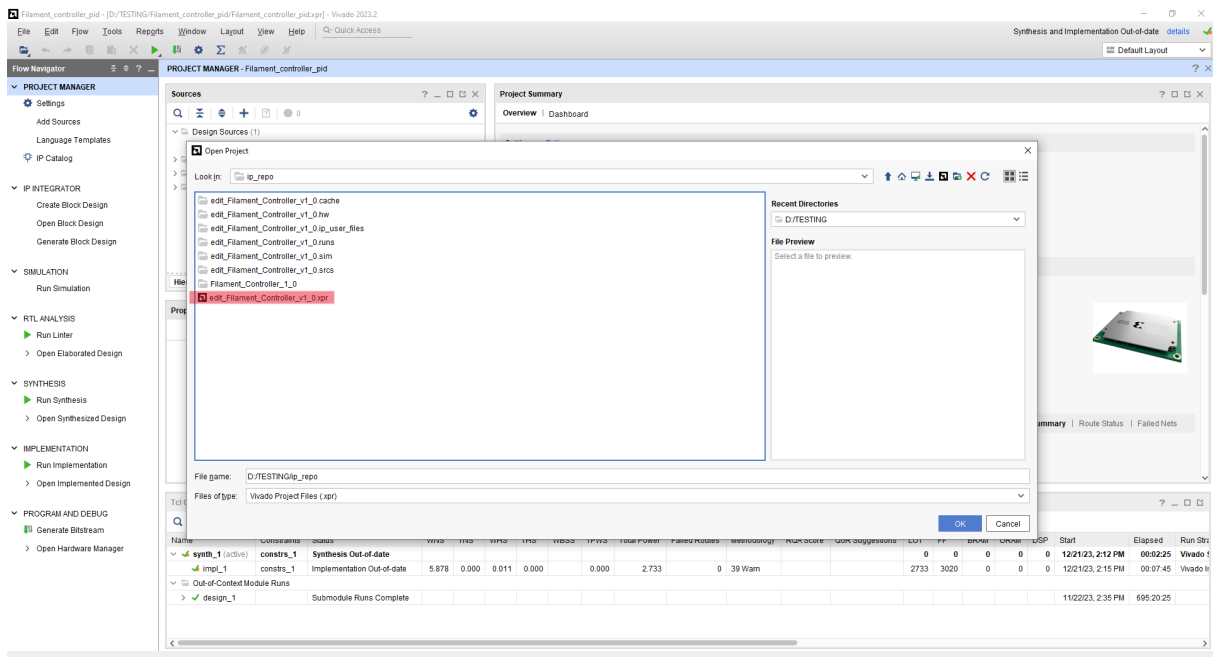


FIGURE 6.3
Opening the filament controller

Then click on the arrow near Filament_Controller_v1_0 to expand the file and reveal Filament_Controller_v1_0_S00_AXI_inst Figure 6.4 and double click on it to open it.

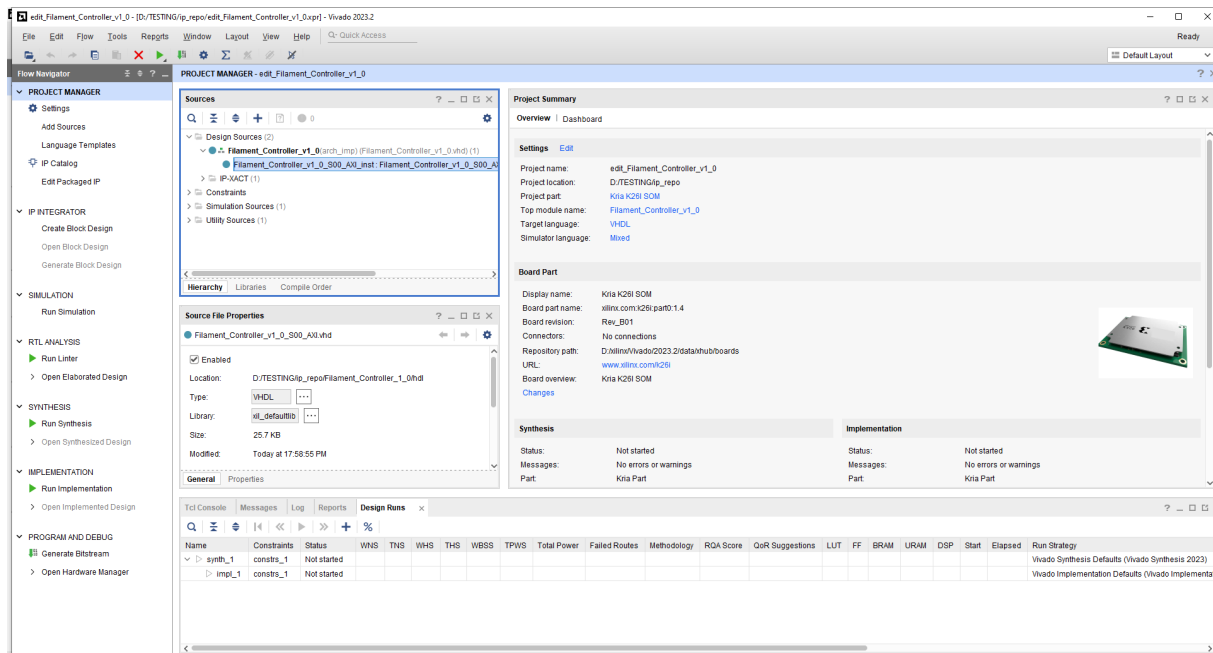


FIGURE 6.4
Opening the filament controller axi interface

On the opened file go to line 245 where there is the code un-commented for 180° phase shift Figure 6.5. The code for 135°, 90°, 45°, 10° and 0° phase shifts are commented. To chose one of the phase shift don't forget to uncomment the previous one. Only one of the 245 to 250 line should be un-commented. If

another phase shift is needed the calculation to perform is : $x = \frac{angle \times 3333}{360}$ where x is the number to write like : "constant pph : natural := x;"

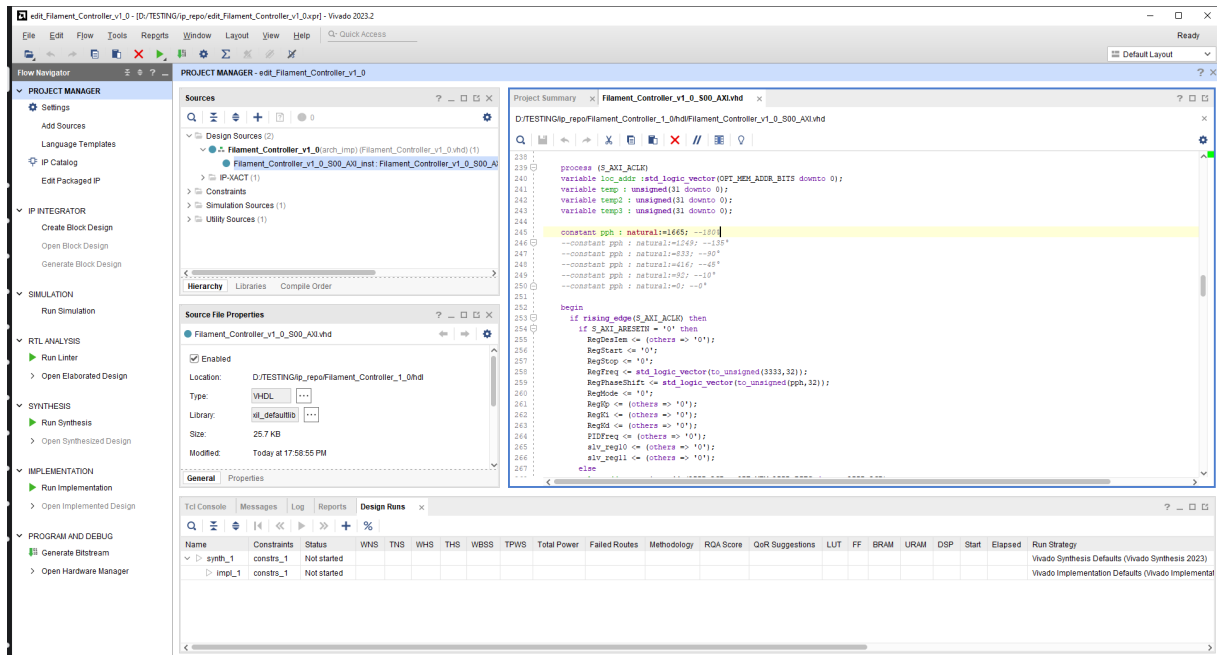


FIGURE 6.5
Line 245 of the filament controller

Then to update the interface, click on the Edit Packaged IP on the right of the screen under the flow navigation Figure 6.6.

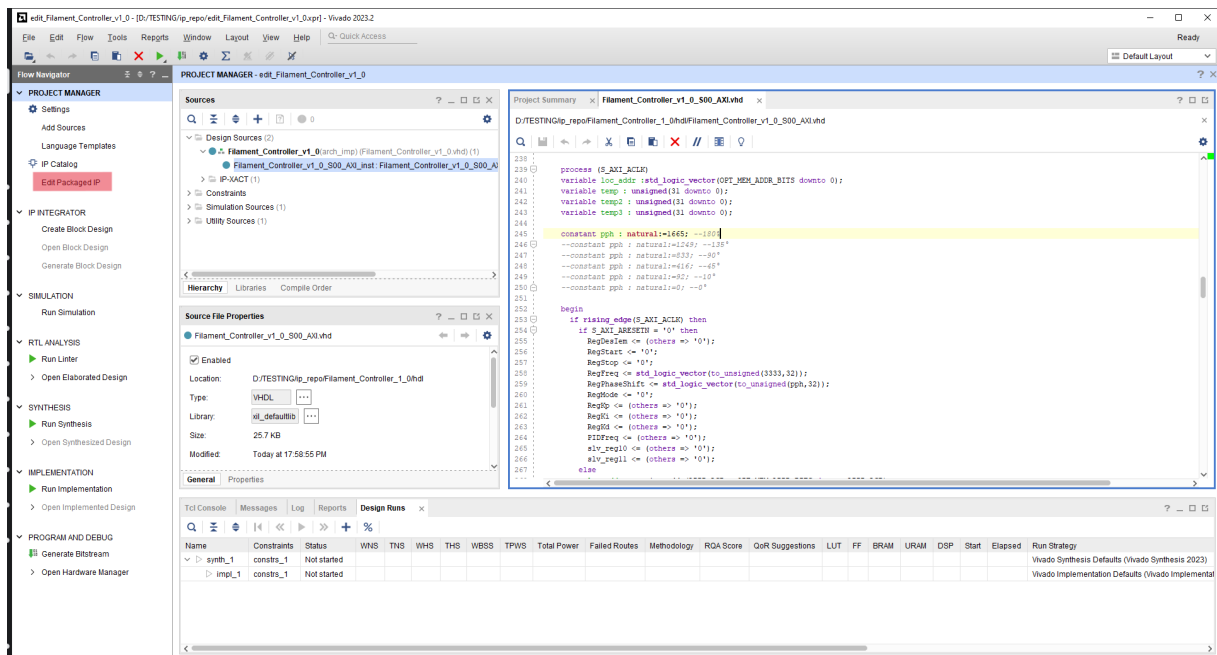


FIGURE 6.6
Edit packaged IP

Then on the newly opened window, click on Review and Package → Re-package IP Figure 6.7.

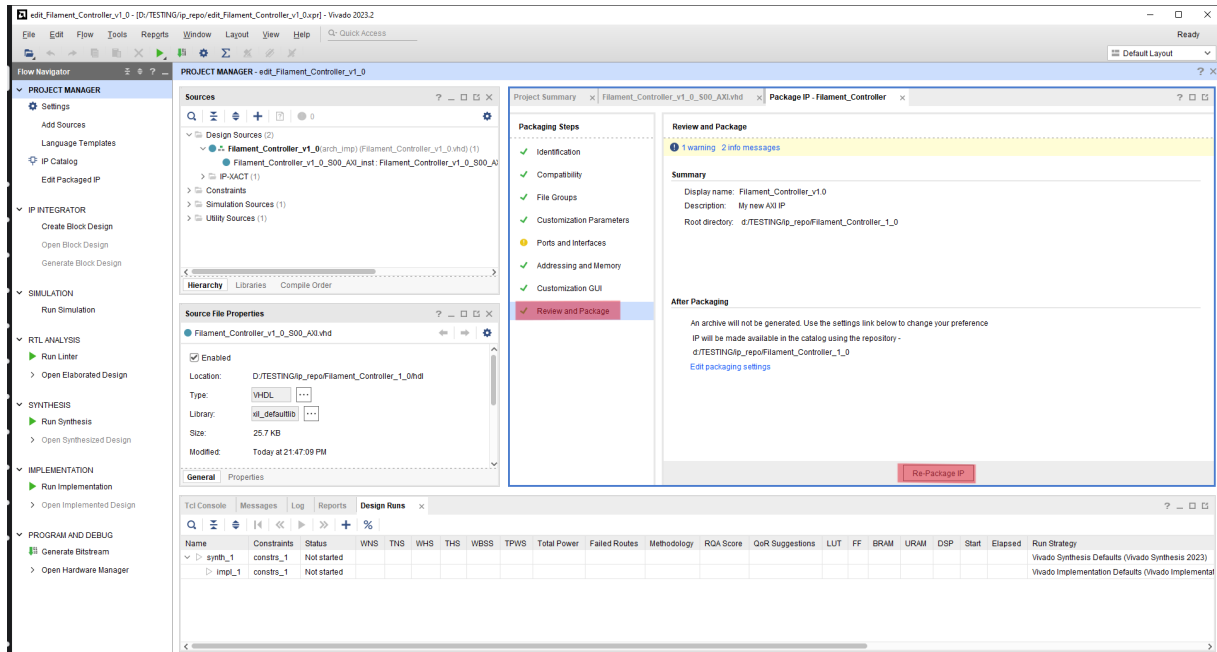


FIGURE 6.7
Re-Package IP

When this is done go back to the main project `Filament_controller_pid`. In this window a ribbon should appear saying IP catalog is out-of-date refresh IP catalog. Click on refresh IP catalog → the filament controller → Upgrade Selected Figure 6.8 (don't forget to click on the filament controller it should be highlighted like in the picture).

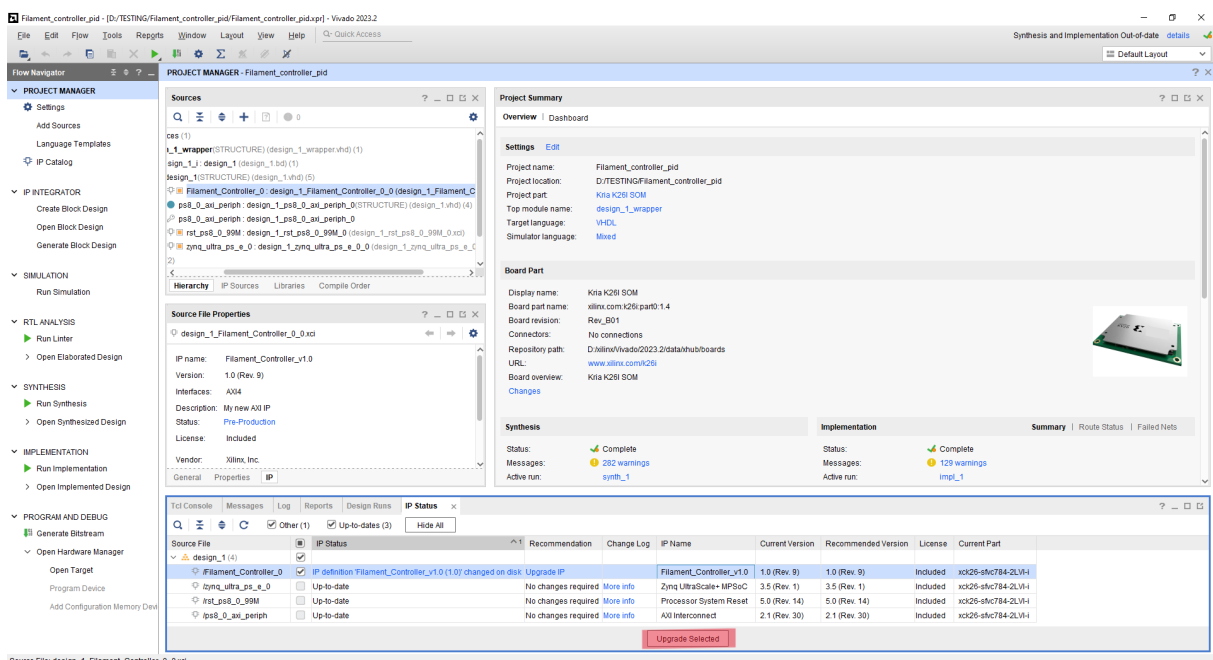


FIGURE 6.8
Upgrade IP

6.1.3 PROGRAMMING THE FPGA

After this, click on generate Bitstream → Yes → Ok and wait for the bitstream to generate (click cancel when asked to open the implemented design). To program the FPGA connect it to the computer and click on open target and on the opened window program the target with the generated bitstream. This must be done each time the phase shift changes.

BIBLIOGRAPHY

- Föhn, Martina et al. (2021). ‘Description of the Mass Spectrometer for the Jupiter Icy Moons Explorer Mission’. In: *2021 IEEE Aerospace Conference (50100)*, pp. 1–14. DOI: 10.1109/AERO50100.2021.9438344.
- Fausch, Rico G., Claudio Zimmermann et al. (2023). ‘Monitoring Space Weather with a Sensitive 1 U CubeSat Mass Spectrometer’. In: *2023 IEEE Aerospace Conference*, pp. 1–11. DOI: 10.1109/AERO55745.2023.10115572.
- Fausch, Rico G., Gregor Moeller et al. (2022). ‘CHESS: Measuring the Dynamics of Composition and Density of Earth’s Upper Atmosphere with CubeSats’. In: *2022 IEEE Aerospace Conference (AERO)*, pp. 01–13. DOI: 10.1109/AERO53065.2022.9843791.